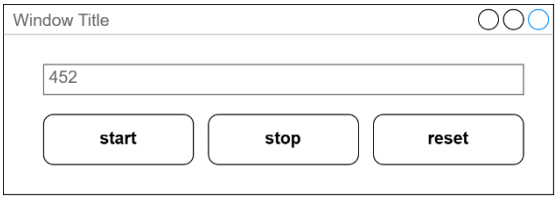
Programowanie obiektowe

Java Swing

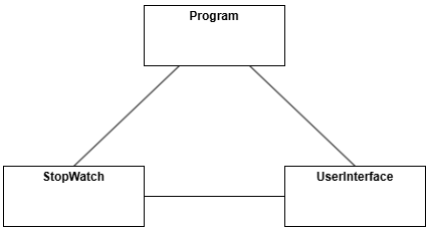
Aby praktycznie poznać czym jest programowanie obiektowe napiszemy program będący małym wyzwaniem. Program ma być w języku Java. Ma wyświetlać na ekranie okno z trzema przyciskami, odpowiednio nazywającymi się start, stop i reset. Nad nimi ma być pokazany w sekundach czas, jaki upływa od chwili naciśnięcia start do chwili naciśnięcia stop. Naciśnięcie reset ma zerować ten czas. Chcemy użyć Swing do tego celu, bo to wydaje się najprostsze. Program nazwijmy StopWatch.



Rysunek 1. Planowany wygląd graficznego interfejsu użytkownika (rysunek zrobiony programem draw.io).

Naturalne jest pytanie: dlaczego po prostu nie zrobimy wszystkiego w jednej klasie, która wyświetla graficzny interfejs użytkownika (GUI)? W latach dziewięćdziesiątych popularne było rapid application development (RAD), czyli przeciąganie kontrolek na formularz a potem dopisywanie do nich odrobiny kodu. Wydawało się to świetnym rozwiązaniem. Program rzeczywiście powstawał szybko. Był jednak zlepkiem funkcjonalności poprzyczepianych do GUI. Mściło się to, gdy program stawał się większy i pojawiała się potrzeba wprowadzenia zmian. Oszczędność czasu była pozorna.

Jeżeli starannie chcemy podejść do naszego projektu, to musimy przemyśleć co chcemy zrobić zanim zaczniemy pisać kod źródłowy. Moglibyśmy rozrysować diagram przypadków użycia, albo nawet formalnie rozpisać te przypadki użycia szczegółowo, ale tym razem, dla tak prostego programu, nie jest to konieczne. Natomiast celowym będzie naszkicowanie diagramu klas, który poglądowo pokaże nasz pomysł na architekturę programu.



Rysunek 2. Szkic diagramu klas UML.

Jak widać mamy trzy klasy (nie licząc bibliotecznych). Zgodnie z zasadą pojedynczej odpowiedzialności (SRP), staramy się rozdzielić pracę pomiędzy każda klasa miała określone kompetencje. I tak klasa Program odpowiada za rozruch całego programu. Klasa StopWatch odpowiada wyłącznie za pomiar czasu. Klasa UserInterface odpowiada za interfejs graficzny.

Czego chcemy od klasy Program? Ma umożliwić uruchomienie programu. Ma utworzyć obiekt klasy Program, aby można było wywoływać metody niestatyczne. Ma utworzyć obiekty klas StopWatch i UserInterface oraz dopilnować, aby istniały wtedy, gdy są potrzebne. Ma również pomóc w

nawiązaniu współpracy pomiędzy tymi obiektami. Wreszcie ma zadbać o uruchomienie obiektu stopWatch klasy StopWatch i utrzymanie jego działania.

Czym ma być klasa StopWatch? Obiekt klasy StopWatch ma po prostu umieć odmierzać czas. Powinien udostępniać metodę getValue(), która zwraca liczbę sekund, jakie upłynęły od wywołania metody start() do wywołania metody stop(). Dorzućmy jeszcze metodę reset(). To jednak nie wszystko. Klasa StopWatch ma również okresowo wywoływać metodę update() obiektu klasy UserInterface, informując go, że upłynęła kolejna sekunda. W tym celu ma pętlę uruchamianą metodą run(), czyli jest Runnable. Więc pętlę tę można łatwo uruchomić w osobnym wątku by zachować płynność działania.

Klasa UserInterface być może powinna nazywać się SwingGraphicalUserInterface, ale taka nazwa byłaby nieco przydługa. Zadaniem tej klasy jest stworzenie interfejsu graficznego, czyli miejsca do wpisywania liczby sekund oraz trzech przycisków. Jest ona również odpowiedzialna za przechwytywanie naciśnięć tych przycisków i wydawanie odpowiednich poleceń obiektowi klasy StopWatch. Powinna także udostępniać metodę update(), przewidzianą do wywoływania przez obiekt klasy StopWatch, która po prostu aktualizuje wyświetlaną liczbę sekund.

Listing 1. Plik Program.java.

```
package example.stopwatch;

import javax.swing.*;

/**
 * Klasa uruchomieniowa aplikacji.
 *
 * <p>Łączy model stopera z interfejsem graficznym i uruchamia cykliczne
 * odświeżanie widoku. Metoda {@link #main(String[])} jest punktem wejścia
 * aplikacji, a pozostała inicjalizacja odbywa się w niestatycznej metodzie
 * {@link #run()}.</p>
 */
public class Program implements Runnable {
    private UserInterface userInterface;
    private StopWatch stopWatch;

    /**
     * Inicjalizuje elementy aplikacji i rozpoczyna działanie stopera.
     *
     * <p>Tworzy model i interfejs, który przechowuje jego referencję. Model
     * otrzymuje z kolei referencję do interfejsu, aby zlecać mu odświeżanie
     * widoku. Następnie uruchamia pętlę cyklicznych aktualizacji.</p>
     */
    @Override
    public void run() {
        // Utwórz model stopera i interfejs z kontrolkami. Interfejs przechowuje
        // referencję do modelu, a model do interfejsu, by móc zlecać odświeżanie.
        //
        stopWatch = new StopWatch();
        userInterface = new UserInterface(stopWatch);

        // Rozpocznij pętlę cyklicznego odświeżania interfejsu.
        // Bezpośrednie wywołanie run() blokuje bieżący wątek.
        //
        stopWatch.run();
        //
        // Ponieważ StopWatch implementuje Runnable, można uruchomić go osobno,
        // jak pokazano poniżej. Powstaje w ten sposób nowy wątek do tego celu.
        //
        //      Thread stopWatchThread = new Thread(stopWatch);
        //      stopWatchThread.setDaemon(true);
        //      stopWatchThread.start();
```

```
    }

    /**
     * Punkt wejścia aplikacji: ustawia skalę interfejsu, wybiera jasny wygląd
     * FlatLaf, a następnie tworzy i uruchamia obiekt konfigurujący aplikację.
     *
     * @param args argumenty wiersza poleceń; nie są używane przez aplikację
     */
    public static void main(String[] args) {
        // Ustaw skalę renderowania elementów Swing, wybierz jasny motyw,
        // a następnie utwórz i uruchom obiekt koordynujący aplikację.
        //
        System.setProperty("sun.java2d.uiScale", "2");
        setupLookAndFeel();
        Program program = new Program();
        program.run();
    }

    /**
     * Ustalanie look-and-feel (tzw. laf), czyli jaki mają wygladać kontrolki -
     * czy mają przypominać te znane z MS Windows, czy raczej takie jakie są na
     * komputerach Apple, czy może jeszcze inne?!
     */
    private static void setupLookAndFeel() {
        // Tablica zawierająca preferowane i tablica zawierające dostępne LaF.
        // Są one final, bo nie będą modyfikowane po utworzeniu.
        //
        final String[] preferredNames = {"Windows", "Nimbus"};
        final UIManager.LookAndFeelInfo[] installed =
UIManager.getInstalledLookAndFeels();

        // Negocjowanie jaki LaF ma być użyty - ponieważ dostępnych LaF jest
        // niewiele (kilka, może kilkanaście) i niewiele jest preferowanych LaF
        // - to użycie dwóch pętli for (jak poniżej) nie jest aż tak złe... jak
        // mogłoby się to wydawać. Przechwytywanie wyjątków (try-catch) jest
        // konieczne, ale - poza odnotowaniem że coś się dzieje - nie wymaga
        // szczególnych kroków - w najgorszym razie nic się nie uda i pozostanie
        // standardowy wygląd kontrolek - co jest dobrym rozwiązaniem.
        //
        try {
            for (String bestName : preferredNames) {
                for (UIManager.LookAndFeelInfo available : installed) {
                    if (available.getName().equals(bestName)) {
                        UIManager.setLookAndFeel(available.getClassName());
                        return;
                    }
                }
            }
        } catch (Exception exception) {
            // ignorujemy wyjątek
        }
    }
}
```

Jak widać metoda main() klasa Program jest punktem startowym: najpierw ustawia skalowanie interfejsu, aby napisy i kontrolki były lepiej widoczne. Następnie wybiera tzw. Look and Feel (LaF), czyli skórkę kontrolek Swing. Na końcu tworzy obiekt klasy Program, po czym wywołuje jego metodę run(). Po co te run()? Skoro program już działa, dlaczego od razu nie umieścić całej logiki w metodzie main()? Chodzi o opuszczenie świata metod statycznych. Właśnie w metodzie run() powstają obiekty klas StopWatch i UserInterface, które następnie są sprzęgane ze sobą. Metoda run() uruchomia także run() klasy StopWatch. W przedstawionej wersji programu ta ostatnia jest wywoływana bezpośrednio, ale lepsze rozwiązaniem jest uruchomienie stopera w osobnym wątku. Osobny wątek pozwala systemowi

operacyjnemu sprawiedliwiej przydzielać czas procesora pomiędzy różne zadania programu, ale tym razem nie jest to konieczne. W naszym programie klasa Program pełni więc przede wszystkim rolę koordynatora. Sama nie mierzy czasu i nie tworzy interfejsu użytkownika.

Listing 2. Plik StopWatch.java.

```
package example.stopwatch;

/**
 * Model stopera. Przechowuje informację o tym, czy pomiar trwa, zapamiętuje
 * jego granice czasowe oraz okresowo zleca interfejsowi odświeżenie widoku.
 *
 * <p>Pomiar bazuje na {@link System#nanoTime()}, który służy do mierzenia
 * upływu czasu monotonicznego. Wartość zwracana przez {@link #getValue()}
 * jest liczbą całkowitych sekund.</p>
 */
public class StopWatch implements Runnable {
    private boolean running;
    private static final long GRANULARITY = 100; // w milisekundach
    private long begin; // w nanosekundach
    private long end; // w nanosekundach

    public UserInterface userInterface;

    /**
     * Cyklicznie zleca interfejsowi odświeżenie wyświetlanej wartości stopera.
     *
     * <p>Pętla działa do przerwania bieżącego wątku. Między aktualizacjami
     * usypia na czas określony przez {@link #GRANULARITY}. Przerwanie podczas
     * uśpienia kończy pętlę.</p>
     */
    @Override
    public void run() {
        // Zlecaj interfejsowi odświeżanie, a między aktualizacjami usypiaj
        // wątek. Przerwanie bieżącego wątku kończy pętlę.
        //
        while (!Thread.currentThread().isInterrupted()) {
            userInterface.update();
            try {
                Thread.sleep(GRANULARITY);
            } catch (InterruptedException e) {
                break;
            }
        }
    }

    /**
     * Rozpoczyna nowy pomiar od bieżącej chwili. Poprzednia wartość nie jest
     * zachowywana — początek i koniec pomiaru ustawiane są na ten sam moment.
     */
    public void start() {
        // Oznacz pomiar jako aktywny i zapamiętaj aktualny czas jako jego początek.
        // Ustawienie końca na tę samą chwilę oznacza, że początkowy wynik wynosi
zero.
        //
        running = true;
        begin = System.nanoTime();
        end = begin;
        userInterface.update();
    }

    /**
     * Zatrzymuje trwający pomiar i zapamiętuje chwilę jego zakończenia.
```

```
     * Jeżeli stoper już był zatrzymany, metoda nie zmienia stanu ani nie
     * wysyła dodatkowego powiadomienia.
     */
    public void stop() {
        // Zatrzymaj pomiar tylko wtedy, gdy jest aktywny: zapamiętaj jego koniec
        // i od razu zleć odświeżenie widoku, bez czekania na kolejny cykl.
        //
        if (running) {
            running = false;
            end = System.nanoTime();
            userInterface.update();
        }
    }

    /**
     * Zeruje pomiar, ustawiając jego początek i koniec na bieżącą chwilę.
     * Stoper po tej operacji pozostaje zatrzymany.
     */
    public void reset() {
        // Zakończ ewentualny pomiar i ustaw jego początek oraz koniec
        // na tę samą bieżącą chwilę, co daje wyzerowany wynik.
        //
        running = false;
        begin = System.nanoTime();
        end = begin;
        userInterface.update();
    }

    /**
     * Oblicza aktualny lub ostatnio zmierzony czas w pełnych sekundach.
     *
     * <p>Podczas działania różnica liczona jest względem bieżącego czasu,
     * a po zatrzymaniu względem zapamiętanej chwili zakończenia. Dzielenie
     * przez liczbę nanosekund w sekundzie obcina część ułamkową.</p>
     *
     * @return liczba całkowitych sekund od rozpoczęcia pomiaru
     */
    public long getValue() {
        // Ustal przelicznik nanosekund na sekundy i oblicz czas trwania:
        // dla aktywnego pomiaru względem bieżącej chwili, a dla zatrzymanego
        // względem zapamiętanego końca. Dzielenie całkowitoliczbowe odrzuca
        // niepełną część sekundy.

        final long NANOSECONDS_PER_SECONDS = 1_000_000_000;
        long duration;
        if (running) {
            duration = System.nanoTime() - begin;
        } else {
            duration = end - begin;
        }
        return duration / NANOSECONDS_PER_SECONDS;
    }
}
```

Klasa StopWatch jest modelem stopera, czyli miejscem, w którym przechowywany jest stan pomiaru czasu. Zgodnie z zasadą pojedynczej odpowiedzialności nie zajmuje się ona tworzeniem okien, przycisków ani pól tekstowych. Jej zadaniem jest wyłącznie odmierzanie czasu oraz informowanie interfejsu użytkownika, że wyświetlane dane mogą wymagać odświeżenia. Stoper przechowuje kilka prostych informacji: czy pomiar jest aktualnie wykonywany, moment rozpoczęcia pomiaru oraz moment jego zakończenia. Czasy te zapisywane są jako liczby całkowite zwracane przez metodę System.nanoTime(). W przeciwieństwie do zegara systemowego nie interesuje nas tutaj aktualna data ani godzina. Potrzebujemy jedynie wiarygodnie mierzyć upływ czasu pomiędzy dwoma zdarzeniami, a

do tego celu nanoTime() nadaje się bardzo dobrze. Klasa implementuje również interfejs Runnable. Oznacza to, że posiada metodę run(), którą można wykonywać bezpośrednio lub uruchomić w osobnym wątku. Metoda ta zawiera prostą pętlę działającą aż do momentu przerwania wątku. Co pewien czas zleca ona interfejsowi odświeżenie wyświetlanej wartości stopera, po czym na chwilę usypia wątek. Dzięki temu interfejs może aktualizować wyświetlany wynik bez potrzeby nieustannego wykonywania obliczeń.Warto zauważyć, że stoper nie aktualizuje wyświetlanej liczby raz na sekundę, lecz znacznie częściej. Stała GRANULARITY określa odstęp pomiędzy kolejnymi aktualizacjami i wynosi 100 milisekund.

Metoda start() rozpoczyna pomiar. Zapamiętywany jest bieżący moment jako początek pomiaru, a jednocześnie ustawiany jest również jego koniec. Dzięki temu bezpośrednio po uruchomieniu wynik wynosi dokładnie zero sekund. Metoda ustawia także znacznik informujący, że pomiar jest aktywny, oraz natychmiast zleca odświeżenie interfejsu. Metoda stop() zatrzymuje pomiar. Jeżeli stoper rzeczywiście pracuje, zapamiętywany jest moment zakończenia pomiaru, a flaga running zostaje wyłączona. Dzięki temu późniejsze odczyty nie będą już korzystać z aktualnego czasu, lecz z zapamiętanego wyniku. Po zatrzymaniu pomiaru interfejs jest od razu odświeżany, więc użytkownik nie musi czekać na kolejny cykl aktualizacji. Metoda reset() służy do wyzerowania stopera. Niezależnie od wcześniejszego stanu pomiar zostaje zatrzymany, a początek i koniec ustawiane są na tę samą chwilę. Efektem jest wynik równy zero sekund. Podobnie jak w pozostałych przypadkach, interfejs zostaje od razu poinformowany o zmianie.

Liczbę sekund, które upłynęły od rozpoczęcia pomiaru podaje metoda getValue(). Jeśli stoper jest uruchomiony, obliczenia wykonywane są względem bieżącego momentu. Jeśli został zatrzymany, wykorzystywany jest zapamiętany czas zakończenia pomiaru. Otrzymana różnica jest następnie przeliczana z nanosekund na sekundy. Ponieważ wynik ma być liczbą całkowitą, część ułamkowa zostaje odrzucona. Oznacza to, że przez pierwszą sekundę działania stoper pokazuje zero, po upływie jednej sekundy pokazuje jeden, po dwóch sekundach dwa itd.

Listing 3. Plik UserInterface.java.

```
package example.stopwatch;

import javax.swing.*;
import java.awt.*;
import java.lang.reflect.InvocationTargetException;

/**
 * Okno graficzne stopera. Klasa tworzy interfejs Swing, przekazuje polecenia
 * użytkownika do obiektu {@link StopWatch} i wyświetla aktualną wartość stopera.
 *
 * <p>Komponenty Swing nie są bezpieczne przy równoczesnym dostępie z wielu
 * wątków. Dlatego tworzenie okna odbywa się w wątku obsługi zdarzeń Swing
 * (EDT), a aktualizacje wyświetlacza są do tego wątku zlecane.</p>
 */
public class UserInterface {

    private final static int horizontalGrids = 3;
    private final static int insetSize = 5;

    private JFrame frame;
    private JTextField display;
    private StopWatch stopWatch;

    /**
     * Tworzy okno, wyświetlacz i przyciski sterujące.
     *
```

```
     * <p>Cała konfiguracja jest wykonywana w EDT. {@code invokeAndWait}
     * sprawia, że konstruktor wraca dopiero po utworzeniu interfejsu,
     * dzięki czemu dalsza inicjalizacja może korzystać z gotowych komponentów.</p>
     */
    UserInterface(StopWatch stopWatch) {
        this.stopWatch = stopWatch;
        stopWatch.userInterface = this;
        try {
            // W EDT utwórz okno, dodaj wyświetlacz i przyciski, a następnie
            // dopasuj rozmiar okna i pokaż gotowy interfejs.
            //
            EventQueue.invokeAndWait(() -> {
                createMainWindow();
                addDisplay();
                addButton("Start", 1, 0, stopWatch::start);
                addButton("Stop", 1, 1, stopWatch::stop);
                addButton("Reset", 1, 2, stopWatch::reset);
                showMainWindow();
            });
        } catch (InterruptedException | InvocationTargetException exception) {
            // Zgłoś błąd inicjalizacji jako wyjątek czasu wykonania, zamiast
            // kontynuować działanie aplikacji bez poprawnie utworzonego okna.
            throw new RuntimeException(exception);
        }
    }

    /**
     * Odświeża wyświetlaną wartość stopera.
     *
     * <p>Odświeżenie jest dodawane do kolejki EDT, zamiast wykonywać operację
     * bezpośrednio w wątku, z którego przyszło powiadomienie. Dzięki temu
     * modyfikacja komponentu Swing odbywa się w bezpiecznym dla niego wątku.</p>
     */
    public void update() {
        // Zleć aktualizację wątkowi EDT, a w nim pobierz bieżącą wartość stopera
        // i ustaw ją jako tekst wyświetlacza. Dlaczego tak? Bo ta metoda będzie
        // wywoływana być może z innego wątku niż EDT.
        EventQueue.invokeLater(() -> display.setText(stopWatch.getValue() + "
seconds"));
    }

    private void createMainWindow() {
        // Utwórz główne okno, ustaw jego tytuł i zachowanie przy zamykaniu,
        // zablokuj zmianę rozmiaru oraz wybierz GridBagLayout jako układ.
        frame = new JFrame();
        frame.setTitle("Stopwatch");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setResizable(false);
        frame.setLayout(new GridBagLayout());
    }

    private void showMainWindow() {
        // Dopasuj rozmiar okna do dodanych komponentów, a następnie je wyświetl.
        frame.pack();
        frame.setVisible(true);
    }

    /**
     * Tworzy pole tekstowe pokazujące czas i umieszcza je w pierwszym wierszu.
     * Pole jest tylko do odczytu, a wyrównanie do prawej ułatwia obserwowanie
     * zmieniających się cyfr.
     */
    private void addDisplay() {
        final int fieldSizeInCharacters = 10;
        display = new JTextField(fieldSizeInCharacters);
        display.setEditable(false);
```

```
        display.setHorizontalAlignment(JTextField.RIGHT);
        GridBagConstraints constraints = new GridBagConstraints();
        constraints.fill = GridBagConstraints.HORIZONTAL;
        constraints.insets = new Insets(insetSize, insetSize, insetSize, insetSize);
        constraints.gridwidth = horizontalGrids;
        constraints.gridx = 0;
        constraints.gridy = 0;
        frame.add(display, constraints);
    }

    /**
     * Dodaje przycisk do wskazanego wiersza i kolumny okna.
     *
     * @param name   tekst widoczny na przycisku
     * @param row    indeks wiersza w układzie siatki
     * @param column indeks kolumny w układzie siatki
     * @param action operacja wykonywana po kliknięciu przycisku
     */
    private void addButton(String name, int row, int column, Runnable action) {
        JButton button = new JButton(name);
        GridBagConstraints constraints = new GridBagConstraints();
        constraints.fill = GridBagConstraints.HORIZONTAL;
        constraints.insets = new Insets(insetSize, insetSize, insetSize, insetSize);
        constraints.gridy = row;
        constraints.gridx = column;
        frame.add(button, constraints);
        button.addActionListener(event -> action.run());
    }
}
```

Klasa UserInterface odpowiada za interfejs graficzny programu. To właśnie tutaj tworzone są okno, pole wyświetlające aktualny czas oraz przyciski Start, Stop i Reset. Klasa ta pełni również rolę pośrednika pomiędzy użytkownikiem a obiektem StopWatch: przechwytuje kliknięcia przycisków, zamienia je na wywołania metod stopera i aktualizuje wyświetlane dane.

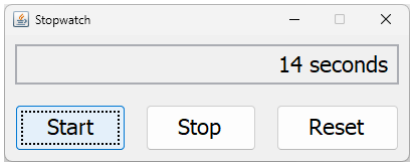
W przeciwieństwie do klasy StopWatch, która przechowuje stan pomiaru i wykonuje obliczenia, UserInterface nie mierzy czasu. Jej zadaniem jest wyłącznie prezentowanie informacji oraz przekazywanie poleceń użytkownika do odpowiedniego obiektu.

Konstruktor klasy otrzymuje referencję do obiektu StopWatch i zapamiętuje ją na później. Dzięki temu przyciski mogą wywoływać metody start(), stop() i reset(). Jednocześnie tworzona jest zależność w drugą stronę: stoper otrzymuje referencję do interfejsu, aby móc zlecać jego odświeżanie. Mamy więc dwa współpracujące ze sobą obiekty, z których każdy zna tylko tyle o drugim, ile jest mu potrzebne do wykonania swojej pracy.

Większość kodu konstruktora zajmuje tworzenie okna programu. Warto jednak zwrócić uwagę na sposób, w jaki jest to realizowane. Swing posiada własny wątek obsługi zdarzeń, nazywany zwykle EDT (Event Dispatch Thread). Komponenty Swing nie są przystosowane do jednoczesnego używania z wielu wątków, dlatego ich tworzenie i modyfikowanie powinno odbywać się właśnie w EDT. Z tego powodu konstruktor nie tworzy okna bezpośrednio, lecz zleca tę operację właściwemu wątkowi i czeka na jej zakończenie. Dzięki temu po zakończeniu konstruktora wiadomo, że wszystkie elementy interfejsu są już gotowe do użycia. Przy tym tworzenie okna zostało podzielone na kilka niewielkich metod pomocniczych. Jedna tworzy główne okno aplikacji, druga dodaje pole wyświetlacza, kolejna dodaje przyciski, a ostatnia pokazuje gotowe okno na ekranie.

Drugą ważną metodą klasy jest update(). Jej zadaniem jest odświeżenie wyświetlanej wartości stopera. Mogłoby się wydawać, że wystarczy odczytać wynik z obiektu StopWatch i wpisać go do pola tekstowego. Problem polega na tym, że metoda ta może zostać wywołana przez stoper działający w

innym wątku. Bezpośrednia modyfikacja komponentów Swing z obcego wątku mogłaby prowadzić do trudnych do wykrycia błędów. Dlatego metoda jedynie zleca aktualizację wątkowi EDT, który wykona ją we właściwym i bezpiecznym miejscu. Warto zauważyć, że klasa UserInterface nie przechowuje własnej kopii mierzonego czasu. Za każdym razem, gdy potrzebuje aktualnej wartości, pobiera ją z obiektu StopWatch. Jest to zgodne z zasadą, że dane powinny mieć jedno główne miejsce przechowywania. Dzięki temu nie pojawia się problem synchronizacji kilku różnych kopii tej samej informacji.



Rysunek 3. Program StopWatch odmierzający czas.

Program działa zgodnie z oczekiwaniami, patrz też rysunek powyżej. Oczywiście można go ulepszyć, dodać więcej funkcjonalności, zmienić wygląd interfejsu użytkownika, zadbać o możliwość obsługi wyłącznie z klawiatury i tak dalej. Ubocznym skutkiem tego co napisaliśmy jest możliwość wygenerowania automatycznie dokumentacji klas programu na podstawie komentarzy narzędziem Javadoc. Zauważmy, Javadoc używa języka angielskiego, przez to otrzymaliśmy dokumentację z pomieszanym tekstem polskim i angielskim, patrz poniżej.



Rysunek 4. Javadoc dla programu.